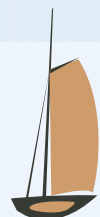


2025年

第5课 质量属性

安全性 可测试性

北京交通大学软件学院



目录

01

案例分析

02

安全性定义

03

安全性战术

04

架构决策

严禁复制

严禁复制

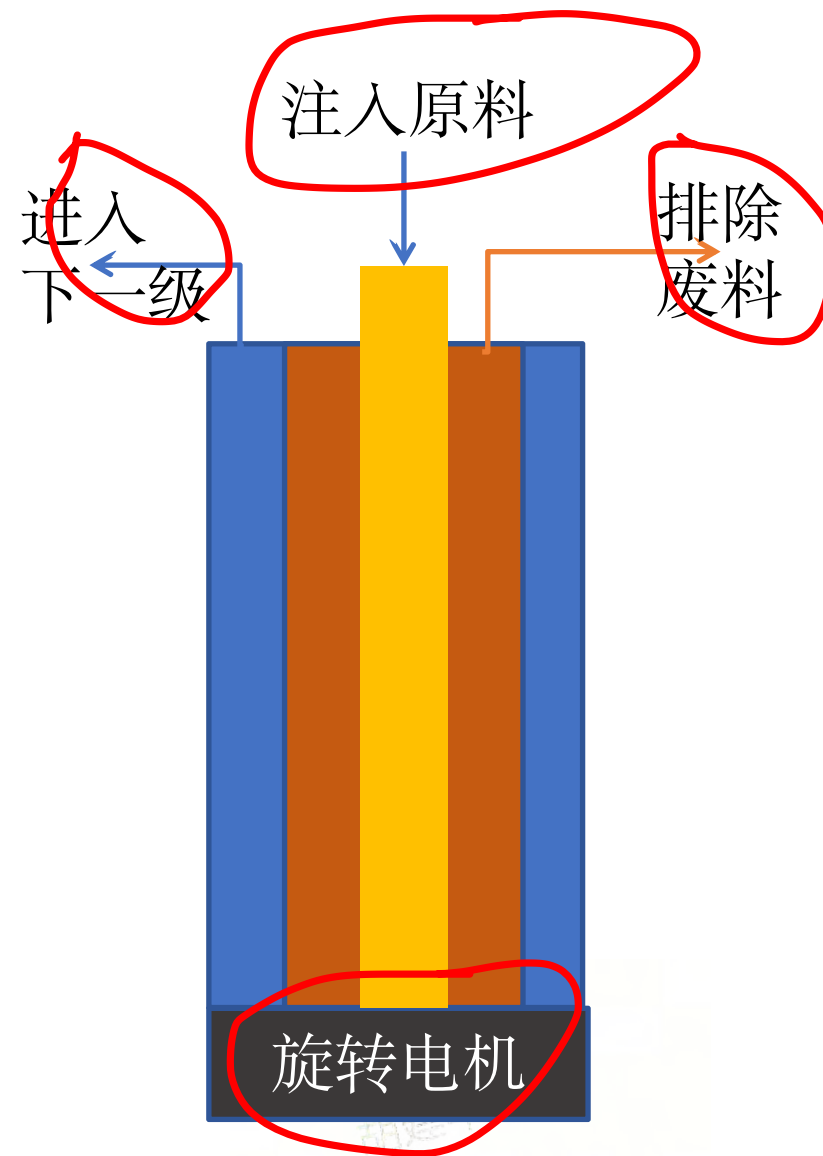
01

案例分析



案例研究Stuxnet

- 离心机，4万-10万转/分钟，1000台，1个月，1公斤



案例研究stuxnet-离心机的控制

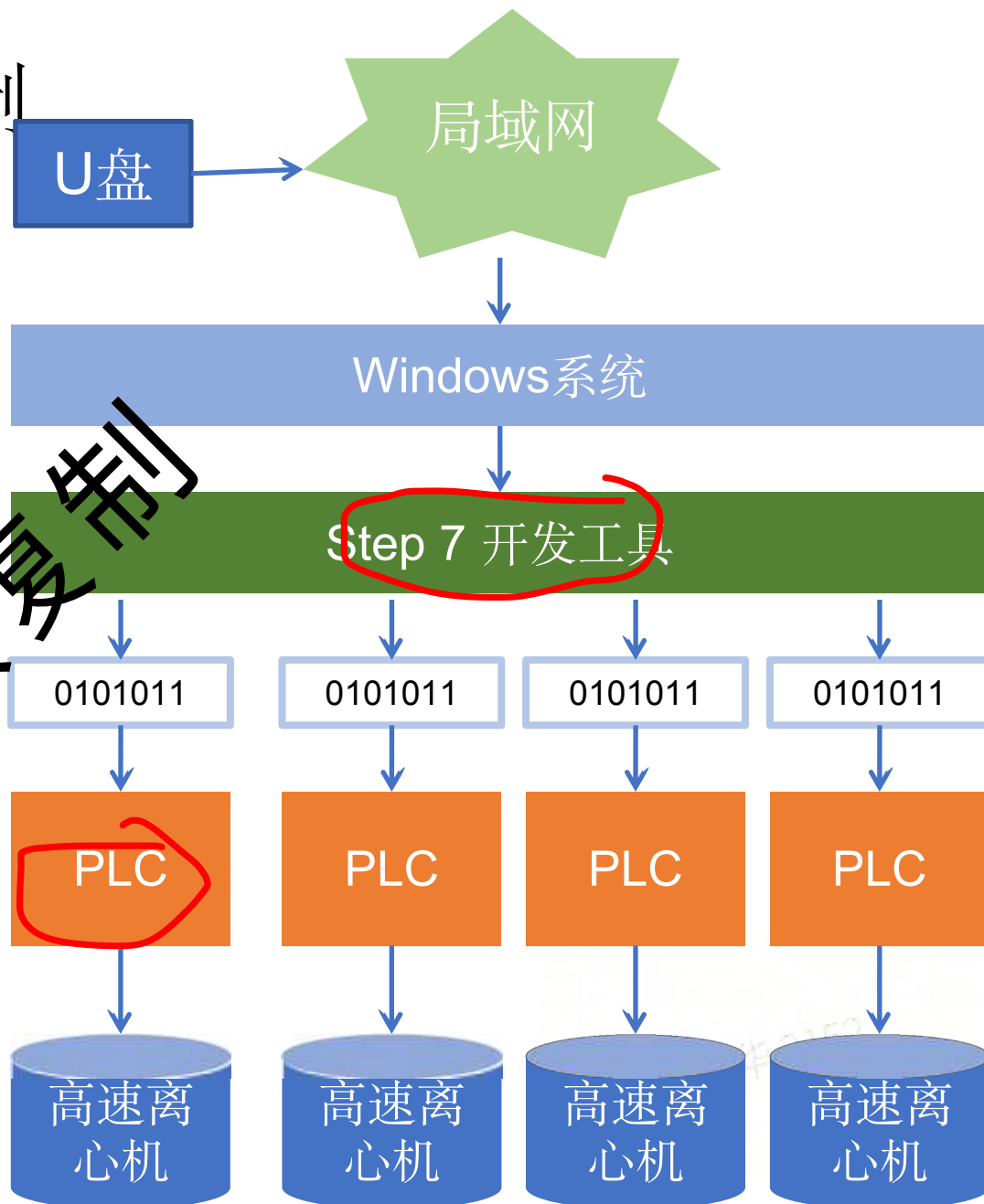
- 病毒感染;
- 修改PLC代码;
- 损坏离心机;
- 破坏产出;

天然铀
U235 0.7%

U235
90%



严禁复制



Stuxnet病毒修改离心机控制代码

预期的控制逻辑:

IF 当前转速 < 100,000 每分钟 :

 转速增加 1000 转每分钟

 RETURN

IF 保持转速时间足够长:

 打开废气阀门

IF 保持转速时间足够长:

 打开浓缩气体阀门

IF 气体已经排除:

 IF 转速 > 0 :

 降低转速

 ELSE:

 打开进气阀门

病毒的攻击手段:

1. 修改传感器的信息;
2. 随意控制阀门;
3. 随意控制转速;

严禁复制

胡建华 2152

02

安全性定义

严禁复制



安全性的3类目标

01

保护数据:

保密性 (Confidentiality) : 只有授权的用户或系统才能查看敏感数据。

完整性 (Integrity) : 确保数据在传输或存储过程中不会遭到非法修改。

02

保护系统:

确保系统不会因攻击或故障导致无法使用。

03

保护用户:

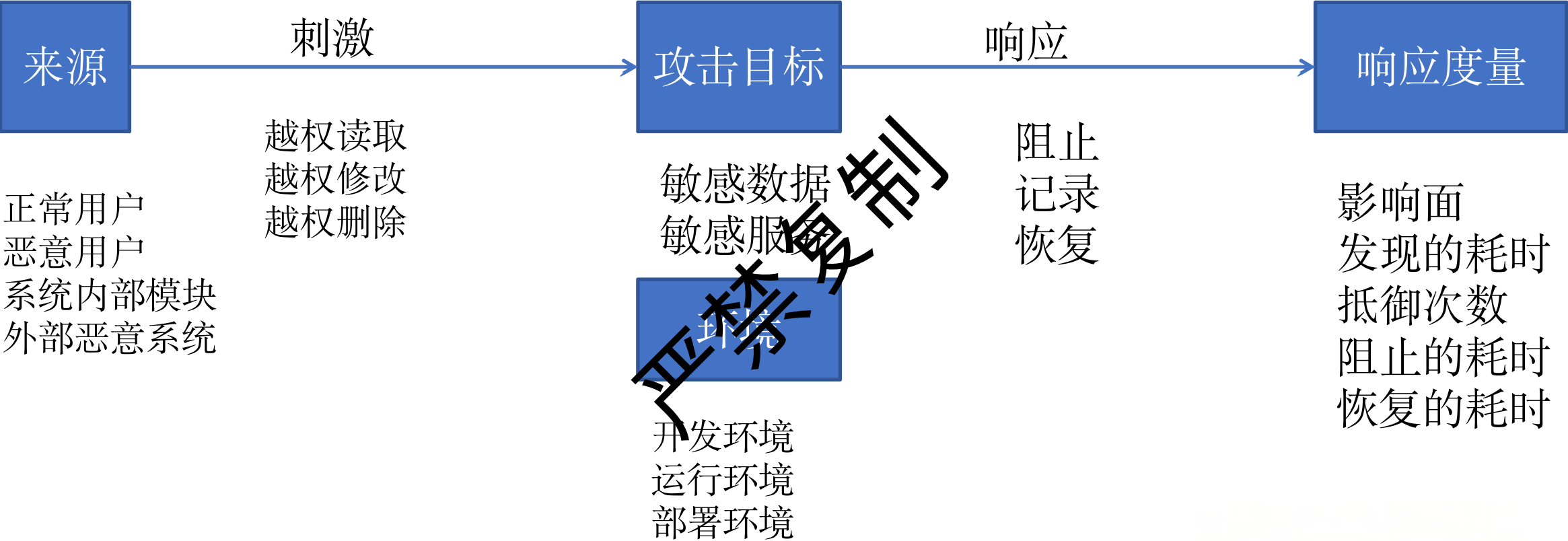
授权 (Authorization) : 支持用户, 使得用户能够执行特定任务。

身份验证 (Authentication) : 保护用户不被冒名顶替。

不可否认 (Nonrepudiation) : 保护用户不会因为对方否认自身行为而被欺骗。

胡建华 2152

安全性的场景



安全性的场景-要素1-刺激

刺激源头

- 未经授权地查看数据。
- 越权修改或删除数据。
- 非法访问系统的服务。
- 改变系统行为。
- 使系统不可用。

刺激目标

- 系统数据服务
- 系统内的数据
- 系统组件
- 系统产生的数据

运行环境

- 在线或离线；
- 互联网或局域网；
- 防火墙配置；
- 系统的运行状态

安全性的场景-要素2-响应

01

响应度量(Response Measure)

被攻击后系统受损程度;
检测到攻击所需的时间;
成功抵御攻击的次数;
从攻击中恢复所需的时间;
攻击导致数据泄露的范围;

02

响应攻击的方式:

拒绝访问;
拒绝篡改;
拒绝虚假身份;
拒绝数据伪造;
拒绝系统瘫痪。

03

跟踪攻击行为:

记录访问或修改。
记录未授权访问尝试。
被攻击时通知管理员。

胡建华 2152

03

安全性战术

严禁复制



两种战术



严禁复制



胡建华 2152

防范用户攻击：预防攻击



验证身份:

- 密码, 证书, 生物识别技术;
- SSO
- OAuth2.0



授权参与者 (**Authorize Actors**)

- Role Based Access Control

严禁复制

RBAC (role based access control)

- 核心思想:
 - 权限授予角色
 - 角色分配给用户
- 主要组成部分
 - 用户: 个体或系统
 - 角色: 职位, 例如: 经理
 - 权限: 操作+资源
 - 资源: 保护对象
- 例子: 一个博客系统:
 - 角色-权限分配:
 - Admin: Create_Article, Edit_Article, Delete_Article, View_Article, Manage_Users
 - Editor: Create_Article, Edit_Article, View_Article
 - Viewer: View_Article
 - 用户-角色分配:
 - Alice -> Admin
 - Bob -> Editor
 - Carol -> Viewer

Alice,Bob,Carol

- 管理员 (Admin)
- 编辑 (Editor)
- 访客 (Viewer)

- 创建文章 (Create_Article)
- 编辑文章 (Edit_Article)
- 删除文章 (Delete_Article)
- 查看文章 (View_Article)
- 管理用户 (Manage_Users)

胡建华 2152

RBAC的优势和挑战

优势:

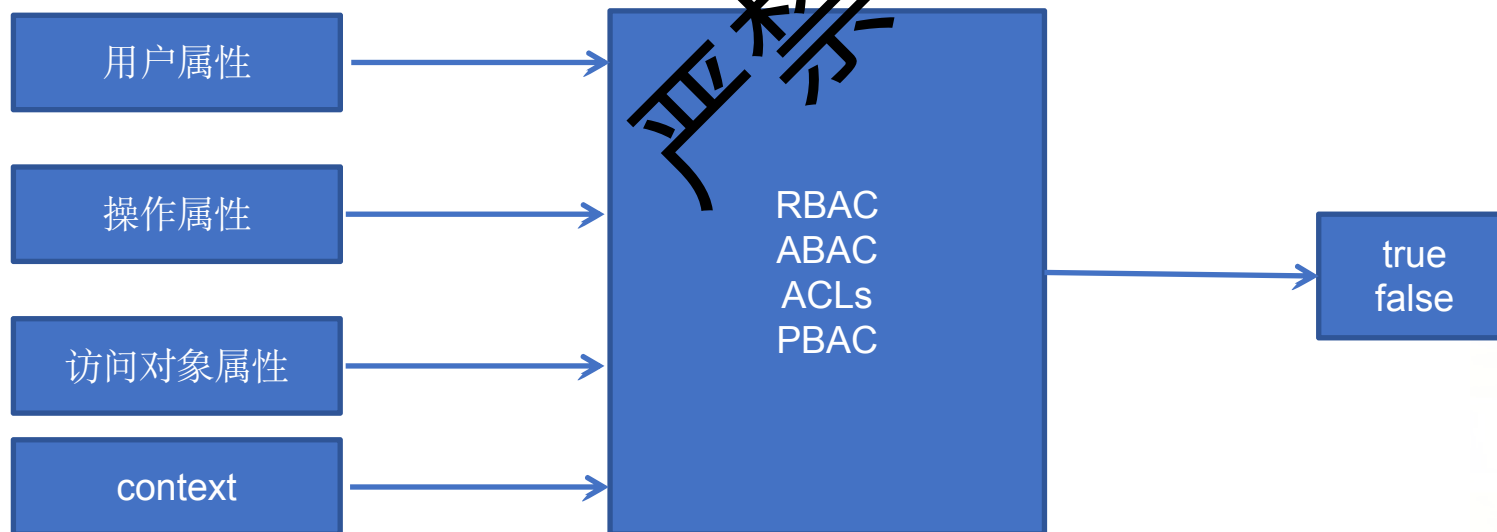
- 易于理解和维护:角色=职位
- 职位变动只需修改角色,无需修改权限。
- 最小权限原则:工作所必需。
- 可扩展性:组织变化,只需要添加新的角色。
- 易于审计:(谁,角色,权限,开始,结束)。

挑战:

- 上下文感知不足(时间,地点,设备)
- 难以控制更细的粒度(字段,实例)
- 权限毒性组合
- 临时增加权限或撤销
- 资源属性相关的授权

RBAC的变体

- ABAC (Attribute-Based Access Control):
 - 用于实现更细粒度、上下文感知的访问控制。
- ACLs (Access Control Lists):
 - 用于控制对特定资源实例的访问。
- Policy-Based Access Control (PBAC):
 - 更广义的策略驱动方法。



ABAC基于属性的访问控制

- ABAC的框架:

- 主体属性: 用户

- 医生 (角色: 心脏病专家、主治医师)、
 - 护士 (角色: 注册护士)、
 - 患者本人。

- 客体属性: 访问对象

- 患者记录 (类型: 病历、化验结果、影像报告、账单信息)、
 - 数据敏感性 (如: 精神健康记录、HIV状况)。

- 操作属性: 操作

- 查看、编辑、注释、分享、删除。

- 环境属性: 环境

- 时间 (工作时间、紧急情况)、
 - 地点 (医院内网、远程访问)、
 - 设备 (授权设备)

- 使用场景:

- “只有当医生的专科 (主体属性) 与患者病历中记录的疾病类型 (客体属性) 相关, 并且是在工作时间 (环境属性) 从医院内网 (环境属性) 访问时, 才允许该医生查看 (操作属性) 患者的详细病历 (客体属性)。”
 - “患者本人 (主体属性) 可以查看 (操作属性) 自己的所有非限制性医疗记录 (客体属性)。”

PBAC

- PBAC (Policy-Based Access Control - 基于策略的访问控制):
 - 核心思想:
 - 策略视为一等公民;
 - 策略本身可以被动态创建、修改、指派和管理。
 - 这些策略能够清晰地描述复杂的授权逻辑。
 - ABAC 可以看作是 PBAC 的一种实现方式。
- 使用场景:
 - 银行卡特殊场景的解决方案 (亲人离世, 子女访问)
 - PBAC 可以通过定义更富有上下文感知和业务逻辑的策略来优雅地处理这种情况。
 - 它允许我们将外部世界的复杂关系 (如家庭关系、法律文件) 和状态 (如账户持有人是否已故) 纳入考量。

Policy ID: DECEASED_PARENT_ACCOUNT_ACCESS
Description: "允许合法子女在父母一方去世后,
Effect: Permit

Target:

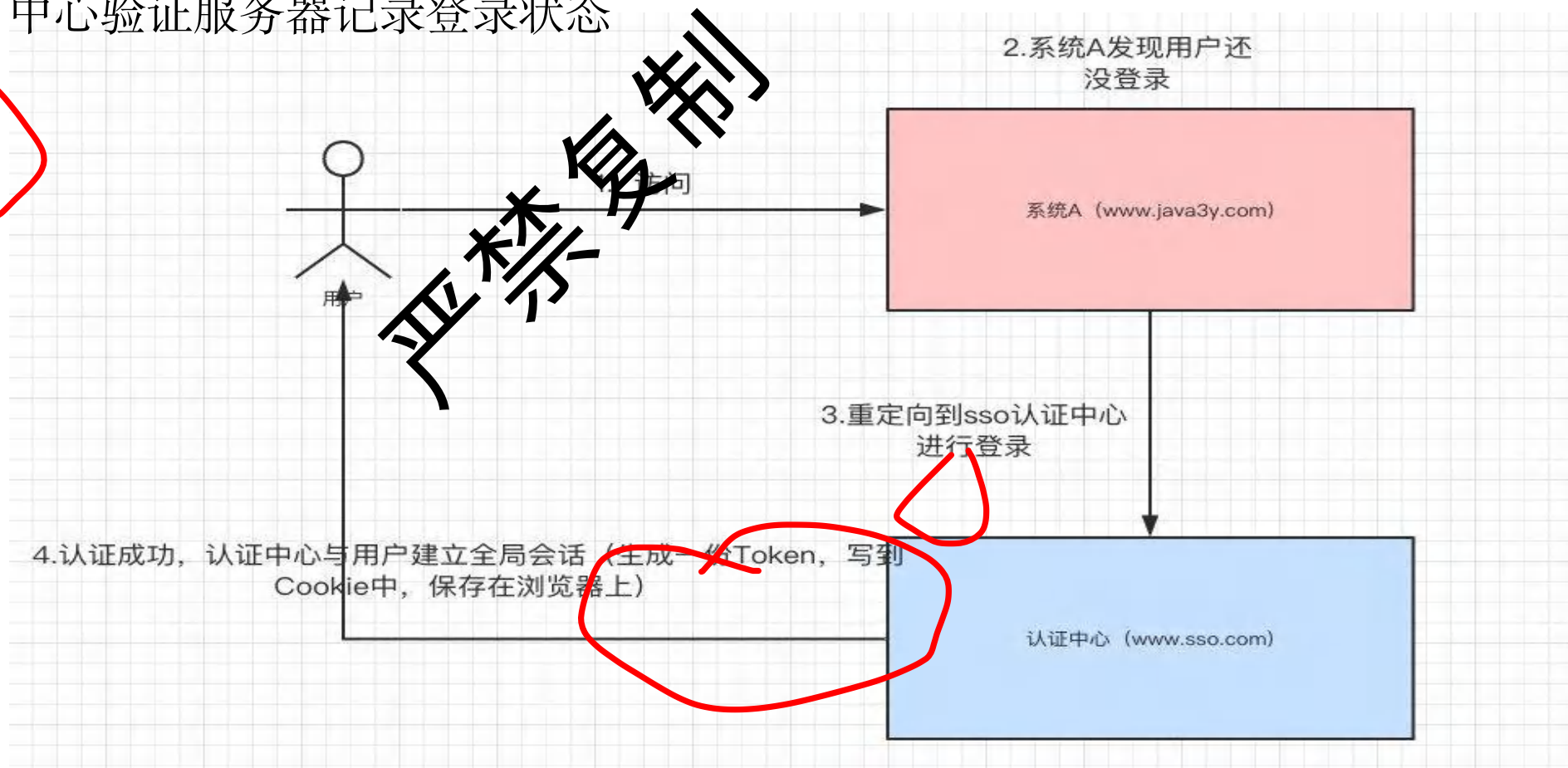
```
Action IN ["view_balance", "request_s  
Resource.type == "bank_account"
```

Condition:

```
AND {  
    // 条件1: 账户持有人已故  
    Resource.owner.status == "deceased"  
  
    // 条件2: 请求者是账户持有人的合法子女  
    // is_legal_child() 可能是一个函数, 它  
    is_legal_child(Subject.id, Resource  
  
    // 条件3: 请求者已提供必要的法律文件证明  
    // has_provided_documentation() 可能  
    has_provided_documentation(Subject.  
    has_provided_documentation(Subject.  
    // (可选) 根据银行规定可能还需要其他文件,  
    // has_provided_documentation(Subje  
}
```

SSO 单点登录

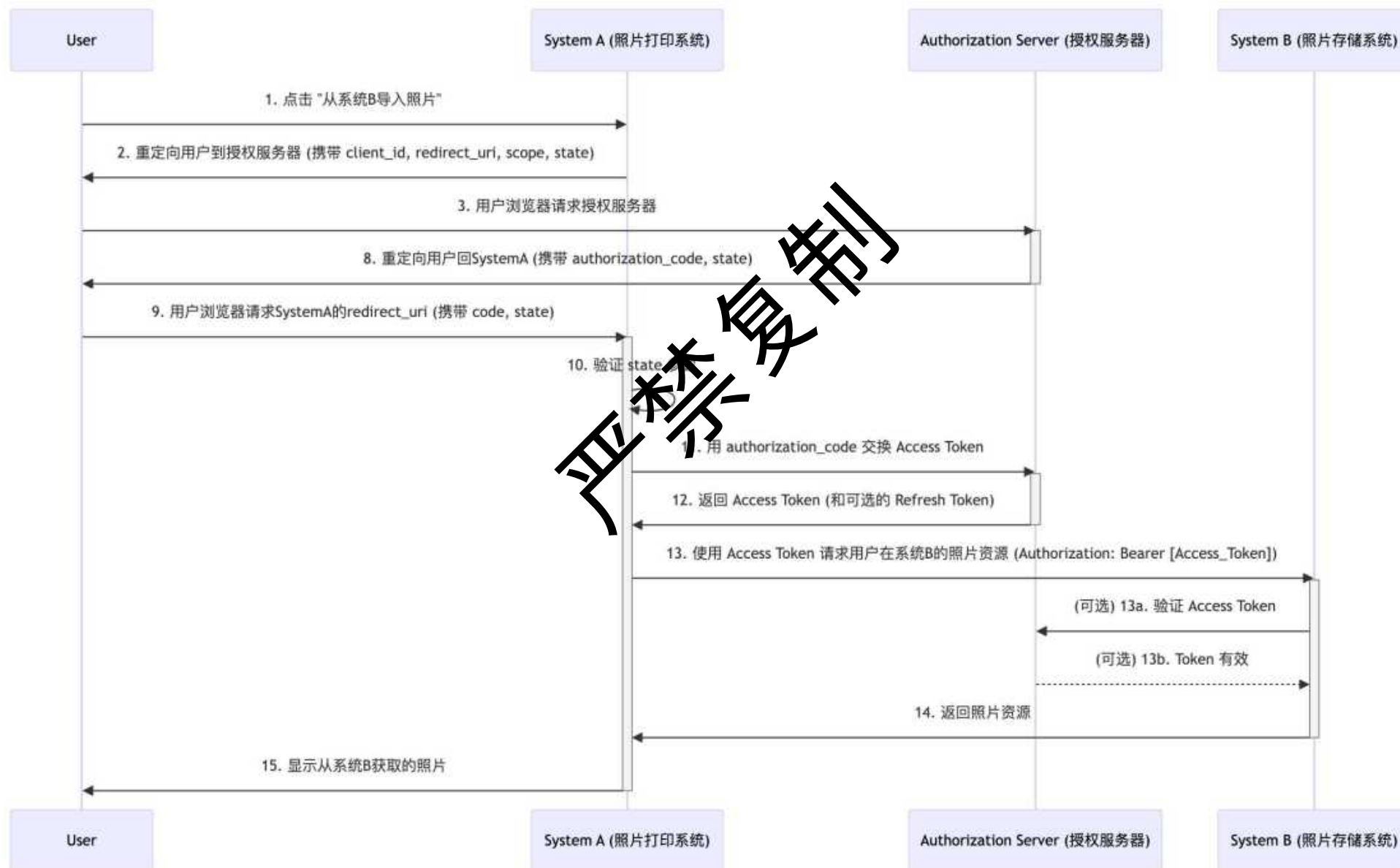
- 如果用户要访问多个系统，重复登录会影响使用
- SSO只需要用户登录一次
- SSO使用CAS 中心验证服务器记录登录状态



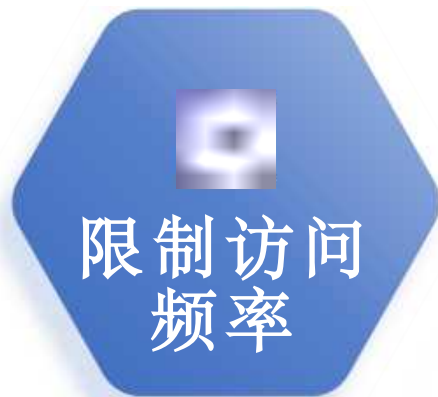
OAuth2.0

- 是一套标准和协议
 - 定义了不同的角色, 授权流程, 令牌类型
 - 有大量的服务实现了OAuth2.0 (如Spring Security OAuth)
- 使用场景:
 - 用户 (Resource Owner): 你。
 - 系统 A (Client Application): 照片打印服务网站
 - 系统 B (Resource Server): 一个云相册服务。
 - 授权服务器 (Authorization Server): 验证用户身份并发出访问令牌。
- 目标:
 - 用户授权系统 A 去访问其在系统 B 中的照片;
 - 不需要将系统 B 的用户名和密码直接告诉系统 A。

OAuth 2.0的实施流程



防范用户攻击：应对攻击



严禁复制

防范系统攻击：发现攻击

01

入侵行为检测 (Detect Intrusion)

匹配恶意行为的特定特征或已知模式。例如：
端口，行为特征；

02

服务拒绝检测 (Detect Service Denial)

(Denial-of-Service, DoS) 模式进行匹配，检测是否存在攻击行为。

03

消息完整性验证 (Verify Message Integrity)

验证数据完整性，确保这些内容在传输或存储过程中没有被篡改。

04

消息延迟检测 (Detect Message Delay)

中间人攻击 (Man-in-the-Middle, MITM) 。

胡建华 2152

防范系统攻击：预防

01

减少暴露：

减少暴露面和分散资源，限制潜在损害范围。

02

数据加密：

通过加密保护数据的机密性，防止未经授权访问。

03

分离实体：

物理分离：网络；
逻辑分离：微服务；
敏感数据分离；

04

安全编码规范：

前端编码规范
后端编码规范

胡建华 2152

前端编码规范

防止 XSS (跨站脚本攻击) :

- 攻击行为: 攻击者向网页中注入恶意脚本并窃取用户的敏感信息 (如 Cookies、Token 等)
- 防御措施: 输入过滤, 输入验证, 内容安全政策, 使用框架, 避免动态代码执行

防止 CSRF (跨站请求伪造) :

- 攻击行为: CSRF 攻击是通过伪造用户请求向 Web 应用发送恶意请求
- 使用 token, 同源策略, Cookie 的 same site 属性;

防止 Clickjacking (点击劫持) :

- 攻击行为: 将恶意网站嵌套在透明 iframe 中诱使用户点击, 攻击者可以劫持用户的操作
- X-Frame-Options 为 DENY 或 SAMEORIGIN

后端编码规范

原则:

最小信任原则 + 分层防御 + 持续更新

输入验证与过滤

白名单机制: 仅允许合法字符集, 数据类型/格式校验 (如邮箱、日期)
使用参数化查询 (预编译语句), 避免动态拼接SQL, ORM框架优先

访问控制

身份认证: 强密码策略、多因素认证 (MFA), 授权机制: RBAC (基于角色的访问控制), 最小权限原则, 会话管理: JWT签名校验、Token有效期控制

数据安全

敏感数据加密传输 (HTTPS/TLS); 密码存储: 加盐哈希 (如bcrypt/Argon2)

安全配置与运维

错误处理: 禁止暴露堆栈信息; 依赖库漏洞扫描 (如CVE监控); 定期渗透测试与代码审计

禁止复制

相建华 2152

安全扫描OWASP ZAP

- OWASP ZAP (Zed Attack Proxy)
 - 全球广受欢迎的、免费且开源的
 - Web 应用程序安全测试工具。
- 核心功能 - 攻击代理 (Man-in-the-Middle Proxy):
 - 浏览器和 Web 服务器之间，拦截 HTTP/HTTPS。
 - 能够拦截、检查、修改甚至重放这些请求和响应。
- 主要特性
 - 自动扫描器：主动发送 payload，尝试触发漏洞。
 - 被动扫描器：。
 - 爬虫：自动发现 Web 的所有页面和资源。
 - Fuzzer: 发送大量畸形或非预期的输入数据。
 - 脚本支持: 允许用户编写自定义脚本。
 - API 支持: 方便集成到自动化测试流程 (如 CI/CD) 中。
 - 插件市场: 拥有丰富的插件，可以扩展其功能
 - 报告功能: 生成详细的漏洞报告。
 - 多种操作模式：。



<https://www.zaproxy.org>

胡建华 2152

ZAP 风险top 10

- A01:2021 - 失效的访问控制 (Broken Access Control)
 - 描述: 未能正确实施对用户访问的内容和功能的限制。
- A02:2021 - 加密机制失效 (Cryptographic Failures)
 - 描述: 与加密相关的失败, 通常导致敏感数据泄露或系统受损。
- A03:2021 - 注入 (Injection)
 - 描述: 例如 SQL 注入、NoSQL 注入、OS 命令注入、LDAP 注入等。
- A04:2021 - 不安全设计 (Insecure Design)
 - 描述: 例如缺乏业务风险分析、威胁建模不足等。
- A05:2021 - 安全配置错误 (Security Misconfiguration)
 - 描述: 这通常是由于不安全的默认配置。
- A06:2021 - 易受攻击和过时的组件 (Vulnerable and Outdated Components)
 - 描述: 使用具有已知漏洞的库、框架和其他软件模块。
- A07:2021 - 身份识别和身份验证失败 (Identification and Authentication Failures)
 - 描述: 与用户身份验证和会话管理相关的功能常常被错误地实现。
- A08:2021 - 软件和数据完整性故障 (Software and Data Integrity Failures)
 - 描述: 这是一个新的类别, 关注与软件更新、关键数据和 CI/CD 管道相关的代码和基础设施。
- A09:2021 - 安全日志记录和监控失败 (Security Logging and Monitoring Failures)
 - 描述: 日志记录和监控不足, 再加上事件响应的缺失或无效集成, 使得攻击者能够进一步攻击系统、保持持久性、转向更多系统, 以及篡改、提取或销毁数据。
- A10:2021 - 服务器端请求伪造 (Server-Side Request Forgery - SSRF)
 - 描述: 当 Web 应用程序在获取远程资源时没有验证用户提供的 URL 时, 会发生 SSRF 缺陷。这使得攻击者能够强制应用程序向其可以访问的内部或外部服务器发送精心构造的请求。



Figure 9.3. Security tactics

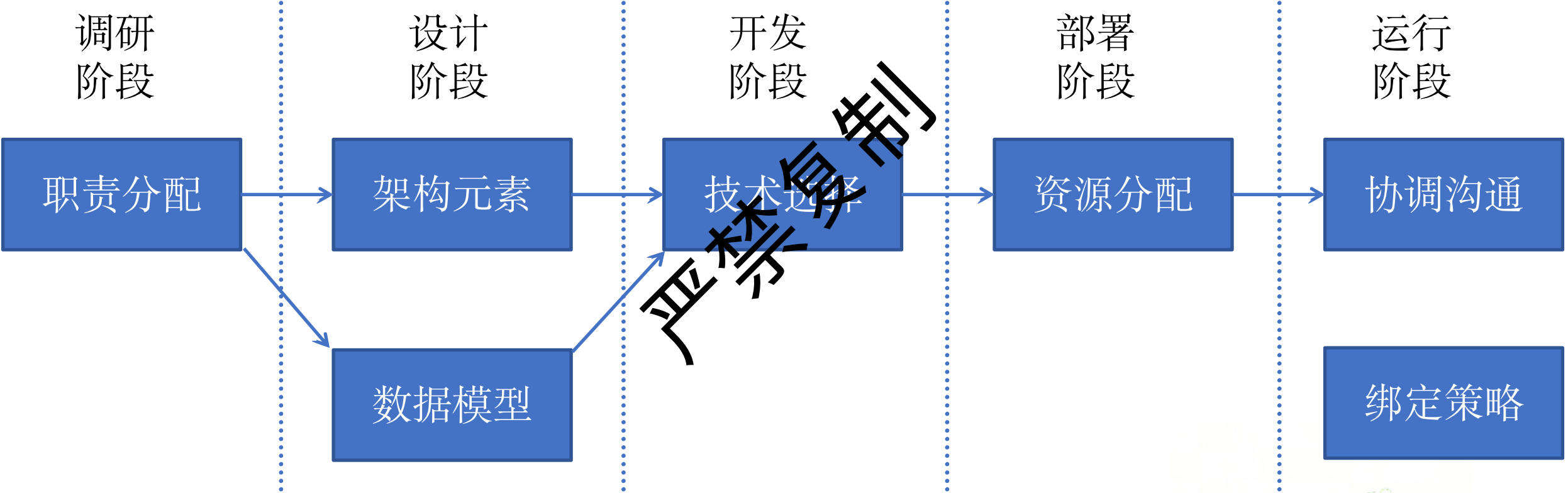
严禁复制

04

架构决策



架构决策概览



职责分配

01

基本安全考虑

基本安全需求，包括识别用户、验证身份、授权访问和记录日志等。只有合法用户能够访问并记录其行为。

02

保障数据安全

数据保护与资源管理，数据加密、校验完整性和监测资源可用性，这些措施旨在保护敏感数据和维持系统稳定。

03

保障系统安全

应对突发情况，系统应具备应急处理能力，例如从攻击中恢复和限制资源使用。

胡建华 2157

数据分级和验证策略绑定

数据分级:

1. 识别并分离不同敏感级别的数据:
 1. 姓名、电子邮件:低敏感;
 2. 信用卡,身份证:高度敏感。
2. 基于敏感级别分配不同的访问权限:。
3. 加密数据并分离密钥与数据:
 1. 敏感数据需要加密存储,
4. 记录敏感数据的访问并保护日志文件。
5. 提供数据恢复机制:
 1. 定期备份数据并设置版本。

01

绑定策略

1. 选择: 开发, 编译时, 发布时, 运行过程
2. 定义:指在系统运行时才动态加载或绑定的组件。
3. 潜在风险:因此在某些情况下带来风险。
例如,某些外部加载的模块或插件,在没有经过严格验证时,可能包含恶意代码。

胡建华 2052
02

架构元素映射 & 协调机制

架构元素映射（内）

1. 确认和认证行为者：识别行为者，认证行为者；
2. 授权与访问控制：授权行为者，授予或拒绝访问；
3. 记录与加密：系统应记录所有访问尝试；所有敏感数据应进行加密处理。
4. 资源可用性管理：识别可用性下降；通知相关人员；限制访问，保证重要的业务。
5. 攻击恢复：快速恢复，减少损失。

01

协调外部系统个人（外）

1. 身份验证与授权：验证通信对方的身份（如通过证书或密钥），第三方支付系统。
2. 加密传输中的数据：使用 HTTPS 协议来加密敏感信息（如支付数据和用户个人信息）。
3. 监控资源使用情况：通信连接数，否为正常流量或潜在的攻击。
4. 限制或终止异常连接：采取限制措施。某个 IP 地址高频访问。

胡建华 20152 02

技术选择

1. 用户认证:

密码认证,多因素认证(MFA),结合多个验证因素,如密码(知识)、手机验证码(拥有)或生物特征(存在)。单点登录(SSO)

2. 数据访问权限管理:

角色基础访问控制(RBAC),
基于属性的访问控制(ABAC):
零信任模型(Zero Trust Model)

3. 资源保护:

硬件、软件和数据资源。常见技术包括:
防火墙(Firewall):
入侵检测和防御系统(IDS/IPS):
访问控制列表(ACLs):
沙箱技术(Sandboxing)。

4. 数据加密:

对称加密:使用相同的密钥进行加密和解密(如AES)。
非对称加密:使用公钥和私钥(如RSA)。端到端加密(E2EE):在传输两端加密数据,确保中间节点无法窃取。哈希算法。

可测试性

严禁复制



汇报人：北京交通大学软件学院

目录

01

案例分析

02

可测试性定义

03

战术分析

04

架构决策

严禁复制

严禁复制

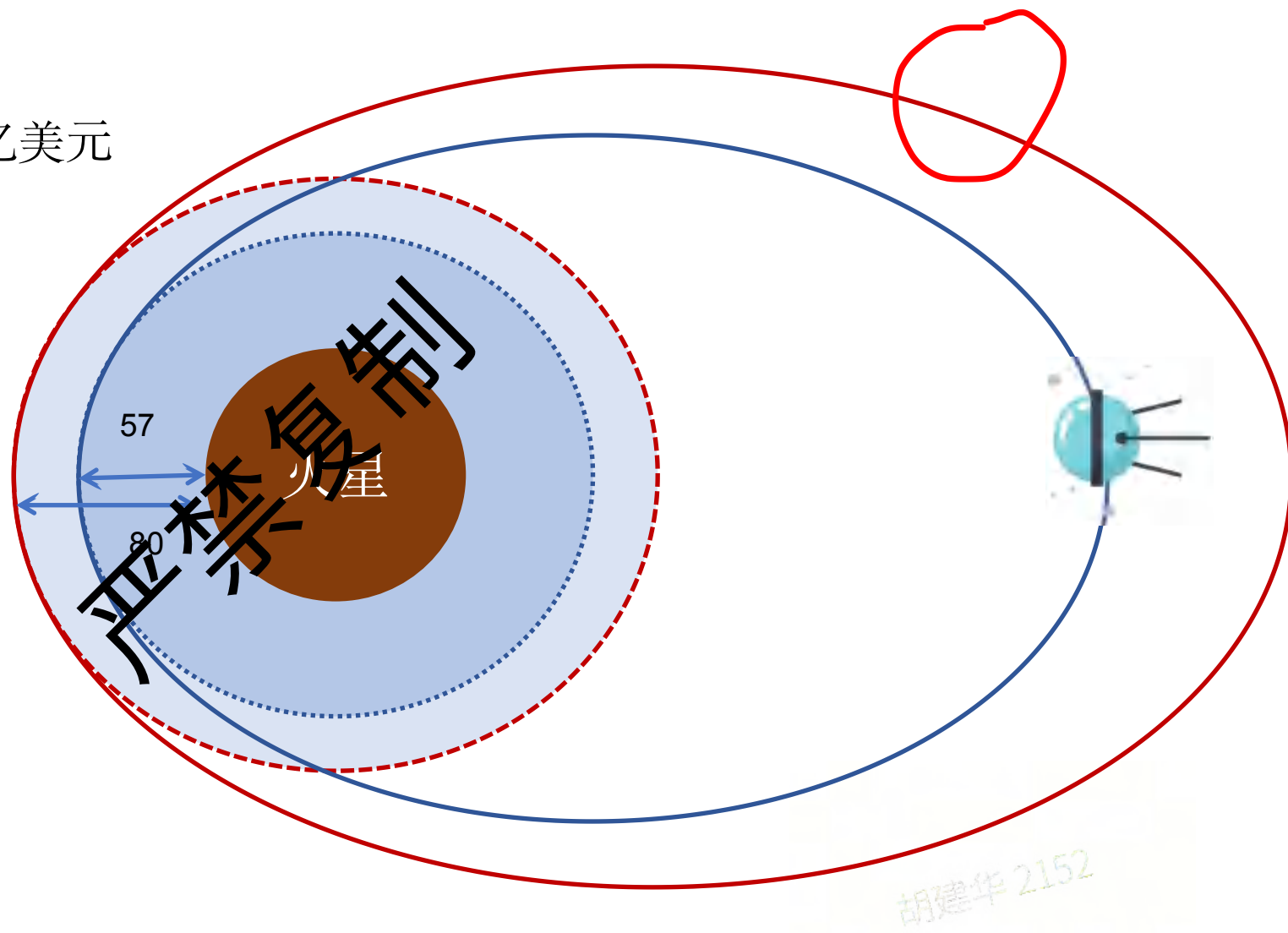
01

案例分析



案例分析

- 1999年火星探测卫星, 3.27亿美元
- 预计轨道近点80KM
- 实际轨道近点57KM
- 大气浓度高烧毁
- 设计方: 牛.秒
- 实施方: 磅.秒
- 1磅力 ≈ 4.48 牛



02

严禁复制

可测试性定义



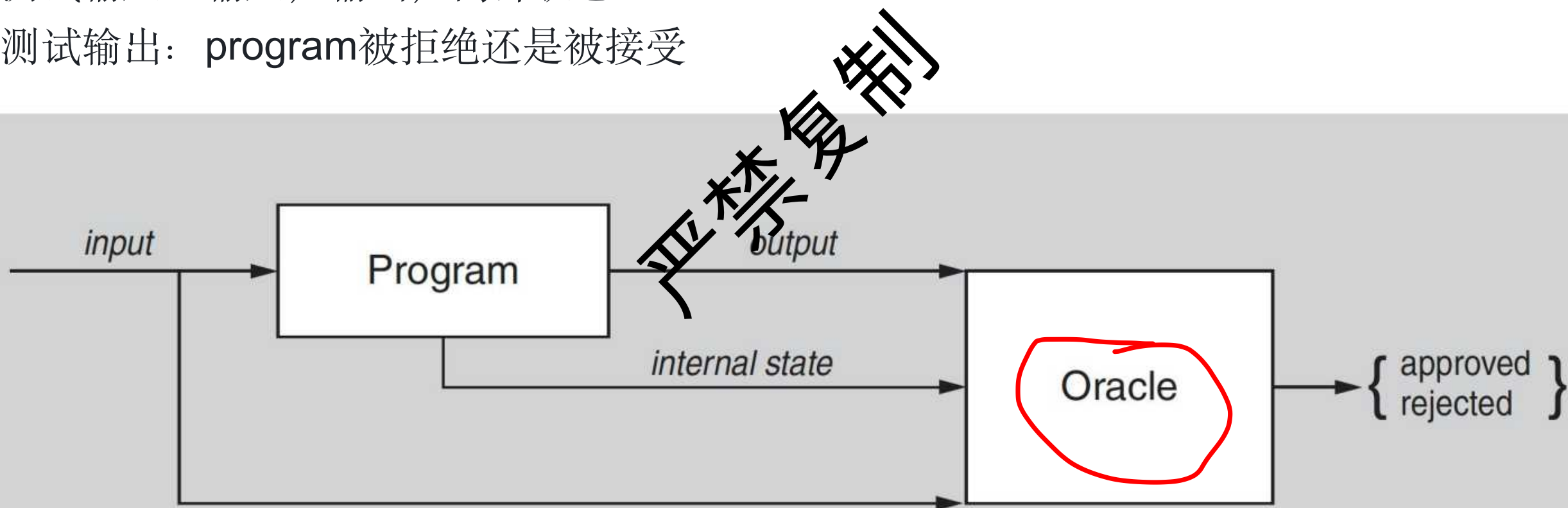
什么是可测试性？

定义：如果测试对象是有错误的，把错误测试出来所需要的相对工作量。

业界评估：30-50%的工作量用于测试。

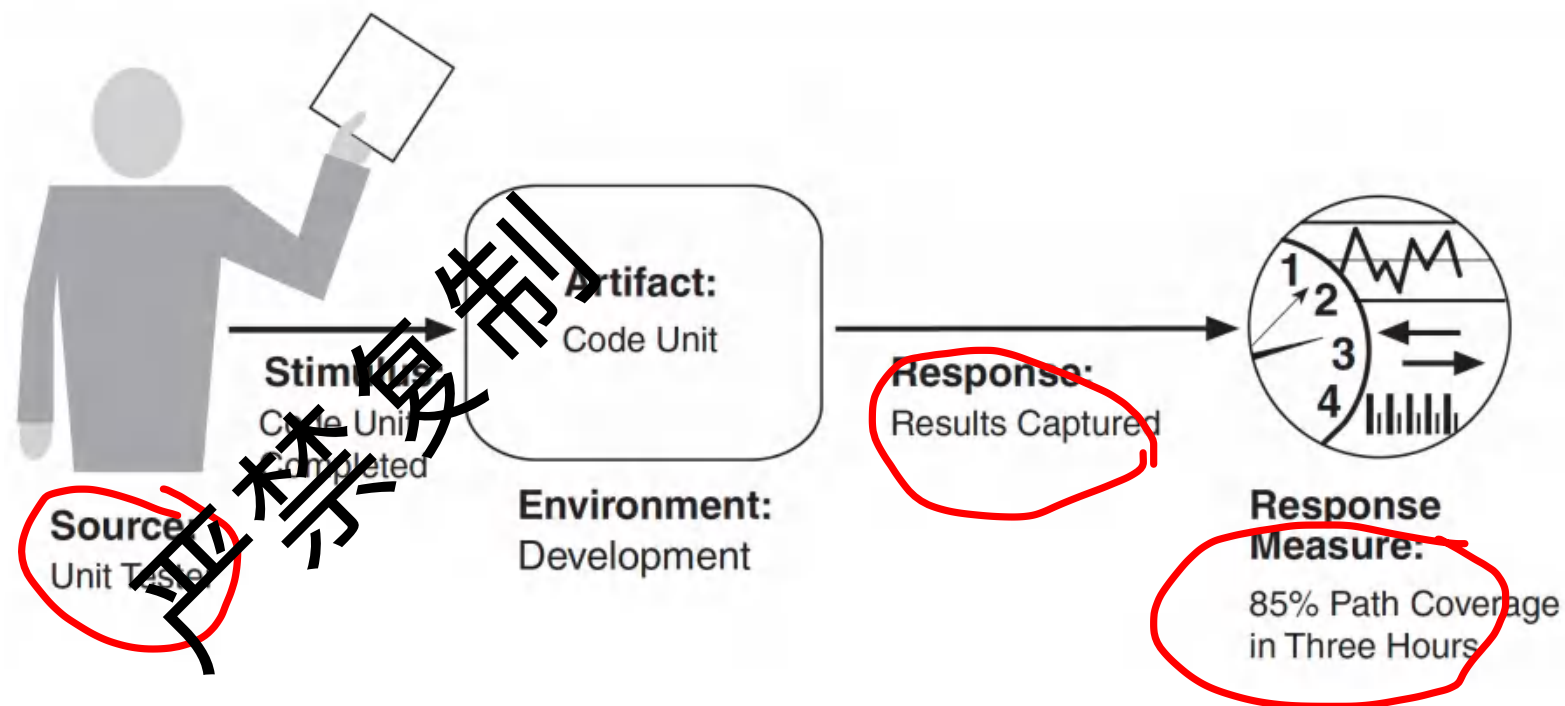
测试输入：输入，输出，内部状态

测试输出：program被拒绝还是被接受



可测试性的场景描述

- 来源
 - 测试, 集成, 用户
- 刺激
 - 测试集合
- 环境:
 - 开发, 设计, 编译,
 - 运行
- 响应
 - 导致错误的行为
- 度量
 - 工作量
 - 执行时间
 - 测试环境准备时间



测试的类型

01

按照测试阶段

1. 单元测试
2. 集成测试
3. 系统测试
4. 验收测试

02

按照测试目标分类

1. 功能测试
2. 非功能测试:
 1. 性能, 安全,
 2. 兼容性, 可用性
3. 维护测试

03

按照测试方法分类

1. 探索性测试
2. 随机测试: 随机输入
3. 流量回放测试
4. 契约测试: 接口测试
5. 混沌测试: 随机故障

混沌测试的例子

Netflix 的猴子军团：对系统进行修改，查看系统的反应。

1. **Chaos Monkey**（混沌猴子）：随机终止运行中的系统进程，看是否崩溃。
2. **Latency Monkey**（延迟猴子）：引入客户端与服务器之间的通信延迟，模拟服务降级。
3. **Conformity Monkey**（一致性猴子）：检测不符合最佳实践的实例，并将其关闭。
4. **Janitor Monkey**（回收猴子）：多余的虚拟机实例会被清理，以节约成本。
5. **Security Monkey**（安全猴子）：未加密的数据库实例，会自动隔离或关闭它。
6. **Localization Monkey**（本地化猴子）：某地区用户无法加载本地化内容，则关闭。

03

可测试性的战术

严禁泄密



测试的代价



前置准备

1. 准备环境
2. 准备系统状态
3. 准备输入



检验输出

1. 检查系统结果状态是否正确
2. 检查输出是否正确



后置动作

1. 恢复环境 (可选)
2. 记录activity



覆盖所有路径

1. 覆盖度报告

可测试性的战术

> 控制和观察类战术

1. 降低前置准备的代价
2. 降低输入准备的代价
3. 降低后置动作的代价
4. 降低记录行为的代价

降低复杂度战术

1. 降低覆盖路径的代价
2. 降低检验输出的代价

严禁复制

控制和观察类战术：环境准备



Sandboxing

- 将系统与实际资源隔离开来，提供一个实验环境。
- 通过虚拟化资源，控制系统时间和外部资源



虚拟化技术：

- Stubs（存根）：用简化模拟版本代替真实组件。
- Mocks（模拟对象）：创建模拟的资源或服务，控制其行为。
- Dependency Injection（依赖注入）：通过注入虚拟或控制的依赖关系来测试系统功能。

控制和观察类战术：初始状态

提供变量的访问和修改

允许访问关键变量、模式或属性。

在电子商务系统中，设置购物车的折扣率，以验证价格计算逻辑。

重置对象的内部状态

重置方法：允许将类或对象的内部状态恢复到指定的初始状态。

在用户账户测试中，可以将用户的登录尝试次数重置为零，用于测试登录次数。

状态实例化：

为了测试系统在任意状态下的表现，最方便的方式是将状态集中存储。

例子：在电子商务平台中，可以存储订单状态，例如“已下单”“待支付”“已支付”“已发货”。

抽象数据源接口：

类似于对系统状态的控制，对输入数据的控制也能让测试更高效。通过抽象接口，可以更轻松地替换测试数据。
例子：电子商务系统在测试推荐算法时，可以用一个虚拟测试数据库，填充特定的用户交易数据。

控制和观察类战术：结果状态

对象状态报告

报告方法：返回对象的完整状态信息，以核对系统内部的运行状态。在订单管理模块中，提供一个方法显示订单的完整状态，包括支付状态、物流信息和商品清单。

启用详细输出或监控

提供方法启用详细输出、事件日志、性能监控或资源使用的检测功能。如：在一个搜索引擎系统中，可以打开性能监控以记录搜索各个阶段的输出。

记录activity:

录制/回放机制:

系统发生故障的状态往往很难重新复现。通过在状态跨越系统接口时记录其状态信息，可以利用这些记录回放系统，从而重现故障。

录制/回放机制包含两个部分：录制：接口日志；回放：使用这些信息作为输入

降低复杂度

1

减少路径

1. 通过断言，断言通常用来检查数据值是否满足指定的约束条件。
2. 通过内聚减少路径

2

降低检验难度

1. 减少组件之间的依赖
2. 通过控制多态：简化对象的继承层次，控制多态性和动态调用
3. 通过控制并发降低检验难度
4. 通过存储正常状态

严禁复制

04

架构决策



可测试性的架构决策

1. 数据模型关注数据抽象:
 1. 识别待测试的主要数据抽象。
 2. 能够捕获数据抽象实例的值,以便进行验证。
 3. 能够设置数据抽象的值,模拟错误,重现故障并进行调试。
 4. 从数据创建到销毁的每个环节都应进行测试。
2. 协调机制:
 1. 支持执行测试套件并记录测试结果;
 2. 支持捕获引发故障的活动;
3. 测试工具
 1. 回归测试框架: JUnit(Java)、pytest(Python)
 2. 服务替代:Mockito,EasyMock,JMock,PowerMock,Spock;
 3. 故障注入工具:Chaos Monkey、Gremlin 等。模拟不同的故障场景,
 4. 录制与回放工具:Selenium 或 TestCafe 等自动化测试工具。